

Relatorio - Sistema de E-commerce em Microservicos

Disciplina: Fundamentos de Computacao Concorrente, Paralela e Distribuida (FCCPD) | Arquitetura: API Gateway + Usuarios + Produtos (2 replicas) + Pedidos | Stack: Python/Flask, comunicacao REST/JSON, JWT (HS256), frontend HTML/CSS.

1. Como a comunicacao entre os microservicos foi implementada?

A comunicacao e feita inteiramente via HTTP/REST com payloads em JSON. O API Gateway e o ponto de entrada unico: ele recebe as requisicoes do cliente e as repassa (proxy) ao microservico responsavel, preservando o metodo, o corpo e o cabeçalho Authorization (JWT). Internamente os servicos tambem conversam por REST: o Servico de Pedidos consulta o Servico de Produtos (GET /products/<id>) para validar o item antes de gravar o pedido, e a replica primaria de Produtos chama a replica par (POST /internal/replicate) para propagar escritas. Optou-se por REST por ser simples, sem dependencias e suficiente para os requisitos; gRPC ou filas (RabbitMQ/Kafka) seriam alternativas para maior desempenho ou desacoplamento assincrono, mas adicionariam complexidade desnecessaria a este escopo.

2. Qual estrategia de consistencia foi adotada na replicacao? Forte ou eventual? Por que?

Foi adotada consistencia FORTE (replicacao sincrona). Quando um produto e criado, a replica primaria grava o dado localmente e so confirma sucesso (HTTP 201) ao cliente apos a replica par confirmar que tambem gravou. Se a propagacao para a par falhar, a escrita local e desfeita e o cliente recebe 503 - ou seja, a operacao e atomica do ponto de vista do cliente (grava nas duas ou em nenhuma). A leitura e distribuida por round-robin entre as duas replicas. A escolha pela consistencia forte garante que qualquer replica lida devolve sempre o mesmo conjunto de produtos, evitando o cliente ver um catalogo diferente a cada leitura. O custo e menor disponibilidade de escrita: se uma replica estiver fora, nao se aceita nova escrita. Consistencia eventual traria maior disponibilidade, mas permitiria leituras temporariamente divergentes - indesejavel para um catalogo de produtos.

3. O que acontece se o Servico de Pedidos cair? O restante continua funcionando?

Sim. Como os microservicos sao independentes e tem bases de dados separadas, a queda do Servico de Pedidos NAO afeta Usuarios nem Produtos: o cliente continua podendo registrar-se, fazer login e navegar/criar produtos normalmente. O heartbeat do Gateway detecta a falha em ate ~10 segundos (2 verificacoes sem resposta no intervalo de 5s), registra a ocorrencia em log com timestamp e passa a responder 503 (Service Unavailable) apenas para as rotas /orders, deixando claro ao cliente qual recurso esta indisponivel. Quando o servico volta, o heartbeat registra a recuperacao e o roteamento e normalizado automaticamente. Isso foi validado em teste: ao derrubar a porta 5003, o Gateway logou FALHA e retornou 503; ao reiniciar, logou RECUPERACAO.

4. Como o JWT garante que um usuario comum nao consiga criar produtos?

No login, o Servico de Usuarios gera um token JWT assinado com uma chave secreta (HS256) contendo userId, email, role (user ou admin) e exp (expiracao). O endpoint POST /products e protegido pelo middleware require_admin, que: (a) valida a assinatura do token com a mesma chave secreta - se o token foi adulterado, a assinatura nao confere e a requisicao e rejeitada com 401; (b) verifica se nao expirou; e (c) exige que o campo role seja exatamente 'admin', retornando 403 caso contrario. Como o role esta dentro do payload assinado, um usuario comum nao consegue forja-lo: alterar o role invalidaria a assinatura. Assim, mesmo possuindo um token valido de usuario, ele recebe 403 ao tentar criar produtos - comportamento confirmado em teste.

5. Quais limitacoes a implementacao possui em relacao a um sistema real de producao?

- Persistencia em arquivos JSON: sem transacoes ACID, indices ou consultas eficientes; nao escala e e

sujeito a concorrência entre processos. Em produção usaria-se um banco real.

- Servidor de desenvolvimento do Flask, não um WSGI de produção (gunicorn/uwsgi) atrás de um balanceador de carga.
- Replicação apenas entre 2 nós, sem eleição de líder, sem reconciliação automática após partição de rede e sem tratamento de escrita quando uma réplica está fora (a escrita falha).
- Comunicação interna em HTTP simples (sem TLS/mTLS por padrão) e chave JWT/segredo de replicação em variáveis de ambiente de exemplo - em produção usar-se-ia um cofre de segredos.
- Sem refresh token, revogação de token, rate limiting, observabilidade (métricas/tracing) nem retry/circuit breaker robustos.
- Heartbeat simples (estado em memória do Gateway, sem persistência nem alta disponibilidade do próprio Gateway, que é um ponto único de falha).

Apesar dessas limitações, o projeto cumpre os objetivos didáticos: decomposição em microserviços independentes, replicação com consistência definida, detecção de falha por heartbeat e segurança entre serviços com JWT.